

Maestría Virtual en Ingeniería de Sistemas y Computación

Gran idea 2: Taller de clasificación de texto y NER

Grandes lenguajes de modelamiento para la tarea de clasificación

Los LLMs (*Large Language Models*) se utilizan actualmente para tareas de procesamiento inteligente de texto. La capacidad contextual de estos modelos supera ampliamente a la de los enfoques de etiquetado secuencial y los algoritmos tradicionales de *machine learning*. Además, una de las tareas fundamentales del Procesamiento del Lenguaje Natural (PLN) es la clasificación de texto, especialmente el análisis de sentimientos. El objetivo de este taller es implementar un clasificador binario y un clasificador de tres clases (análisis de sentimientos), además de implementar un modelo de Reconocimiento de Entidades Nombradas (NER) utilizando LLMs. Cada tarea se divide en cuatro fases:

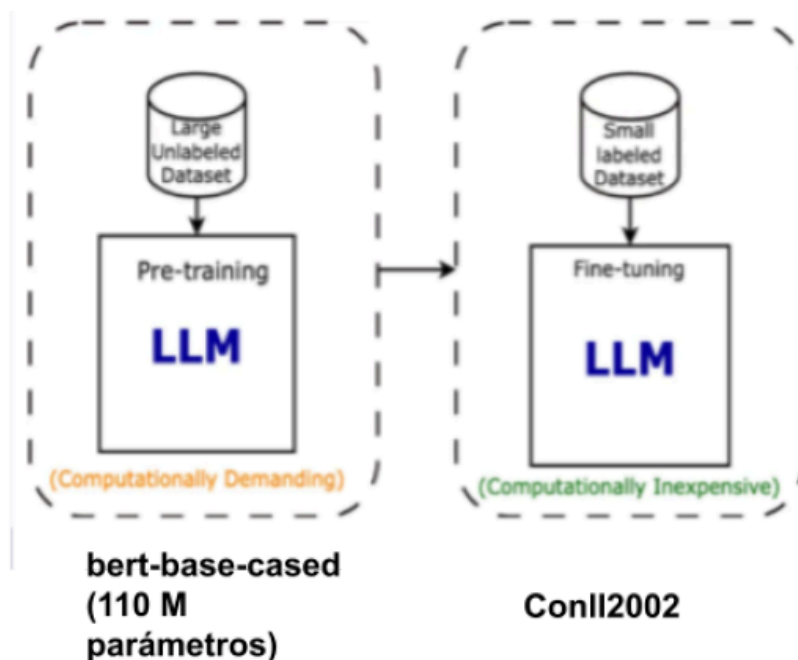
1. Preprocesamiento de los conjuntos de datos,
2. Tokenización,
3. Entrenamiento o *finetuning* del modelo, y
4. Evaluación del desempeño y publicación del modelo en Hugging Face.

BERT

El modelo BERT es una arquitectura *Transformer-encoder* preentrenada sobre un corpus extenso de datos en inglés y multilingüe, mediante aprendizaje auto-supervisado. Esto significa que se entrenó únicamente con texto sin etiquetar (sin intervención humana para generar etiquetas), lo que permite aprovechar grandes volúmenes de datos públicos. Su preentrenamiento se basa en dos objetivos principales:

- **Modelado de lenguaje enmascarado (MLM):** Toma una oración, enmascara aleatoriamente el 15 % de las palabras en la entrada, la procesa completa a través del modelo y predice las palabras enmascaradas. A diferencia de las redes neuronales recurrentes tradicionales (RNN), que procesan las palabras secuencialmente, o de los modelos autorregresivos como GPT, que enmascaran los tokens futuros, MLM permite que el modelo aprenda una representación bidireccional de la oración.
- **Predicción de la siguiente oración (NSP):** El modelo concatena dos oraciones enmascaradas como entradas durante el entrenamiento previo. A veces corresponden a frases que estaban una al lado de la otra en el texto original, a veces no. Luego, el modelo tiene que predecir si las dos oraciones se suceden o no. De esta manera, el modelo aprende una representación interna del idioma inglés que luego puede usarse para extraer

características útiles para tareas posteriores: si tiene un conjunto de datos de oraciones etiquetadas, por ejemplo, puede entrenar un clasificador estándar usando las características producidas por BERT. En la siguiente figura se presenta el esquema de *finetuning* o ajuste, donde a partir de un gran corpus de preentrenamiento se adapta un conjunto de datos anotado con el fin de transferir el modelo preentrenado y resolver una tarea específica de PLN.



Punto 1. Clasificación de texto usando el dataset en español de TASS y los LLMs de XLNet, XLM-RoBERTa

En este ejercicio se deben afinar y entrenar dos conjuntos de datos: [TASS](#), (anotado con tres clases: [NEG](#), [NEU](#), [POS](#)) y [Sarcasmo](#) (anotado con dos clases: [sarcasmo](#) y [no_sarcasmo](#)). Para ello se utilizarán los modelos de clasificación [BETO](#), [XLNet](#) y [XML-Roberta-large](#).

Esta tarea consta de dos subtareas. En la primera, se debe afinar el dataset [TASS](#) con los modelo [BETO](#), [XLNet](#) y [XML-Roberta-large](#) siguiendo estos pasos:

1. Cargar el dataset [TASS](#) de manera local y mostrar la distribución de etiquetas para los tres conjuntos (entrenamiento, validación y prueba).
2. Implementar el ajuste o fine tuning para los modelos [XLnet](#) y [XML-Roberta-large](#) sobre el dataset [TASS](#) considerando todas las fases; preprocesamiento, alineación del dataset, tokenización, padding (mapeo) y el entrenamiento. Como insumo, se comparte el finetuning del modelo [BETO](#) sobre TASS el cual alcanza un f1-score del 64% . La idea es superar éste rendimiento con [XLnet](#) y [XML-Roberta-large](#).

```
trainer.train()
```

[1205/2410 02:57 < 02:57, 6.79 it/s, Epoch 5/10]

Epoch	Training Loss	Validation Loss	F1 Macro	F1 Weighted	Accuracy
1	0.916600	0.840342	0.617039	0.620422	0.622268
2	0.728700	0.809393	0.649775	0.650433	0.650364
3	0.526600	0.874288	0.631498	0.637346	0.643080
4	0.356200	1.043524	0.628428	0.630278	0.624350
5	0.218700	1.270472	0.636006	0.640933	0.640999

De igual manera, se debe afinar y entrenar el dataset [Sarcasmo](#) con los modelos de [BETO](#) y [XML-Roberta-large](#) implementando los mismos pasos del anterior (preprocesamiento, alineación, tokenización, *padding* y el entrenamiento). En este sentido, es importante probar varias combinaciones de *batch size* según la capacidad de la GPU (8, 16 y 32 GB), utilizando GPU de Colab o [Kaggle](#) para ambos afinamientos.

3. Implementar las métricas de desempeño (*f1-score*) para batch size de 8,16, 32 ejecutando entre 6 y 10 épocas con *Early Stopping* para cada uno de los dos afinamientos de [TASS](#) y [Sarcasmo](#) con los respectivos modelos.

4. Los modelos se deben publicar en Hugging face y utilizarse para clasificar las siguientes sentencias:

```
1 === PREDICCIONES DE EJEMPLO ===
2 Positivo 😊 | Me siento muy feliz hoy, fue un día increíble
3 Negativo 😞 | No puedo creer lo mal que salió todo, qué desastre
4 Neutro 😐 | Mañana hay reunión de trabajo a las 10am
5 Negativo 😞 | Odio los lunes, siempre pasa algo horrible
6 Positivo 😊 | Qué bonita sorpresa, no me lo esperaba!
```

Para probar los modelos entrenados con el dataset [Sarcasmo](#), usar los siguientes ejemplos.

Texto: ¡Qué divertido, otro lunes de trabajo! 😞

Predicción: SARCASMO

Confianza: 100.00%

Prob. sarcasmo: 100.00%

Texto: Me encanta cuando llueve el día que tengo planes al aire libre

Predicción: NO_SARCASMO

Confianza: 99.86%
Prob. sarcasmo: 0.14%

Texto: Obvio que aprobé el examen, estudié exactamente 5 minutos
Predicción: SARCASMO
Confianza: 100.00%
Prob. sarcasmo: 100.00%

Texto: Qué sorpresa que el internet se cayó justo cuando iba a entregar la tarea
Predicción: SARCASMO
Confianza: 100.00%
Prob. sarcasmo: 100.00%

Texto: Sí, porque claramente yo soy el experto en cocina gourmet 🍳
Predicción: SARCASMO
Confianza: 100.00%
Prob. sarcasmo: 100.00%

5. Realizar un análisis completo respondiendo:

¿Por qué los rendimientos sobre [TASS](#) son menores que los obtenidos por el dataset [Sarcasmo](#)?

¿Qué papel juegan el número de clases en ambos datasets?

¿Analizar y comprar el desempeño de los LLMs; [Xlnet](#) y [XML-Roberta-large](#) además de [BETO](#) frente a [XML-Roberta-large](#) sobre el dataset [Sarcasmo](#)?

Punto 2. Reconocimiento de entidades nombradas (NER)

Esta tarea consiste en el reconocimiento de entidades nombradas (NER) mediante *Machine Learning* (usando LSTMs) sobre el dataset [Conll2002](#) y mediante *Transfer Learning* usando LLMs bajo el afinamiento (finetuning) de modelos preentrenados tales como [_BETO](#) y [XML-Roberta](#) sobre el dataset [prostata](#). Como insumo para el ajuste y entrenamiento de los LLMs, se proporciona en la carpeta [ayuda](#) un modelo base de [Bilstm-base](#) con un 64.3% de f1-score y el modelo [bert-base-cased](#) con un 75% sobre [Conll2002](#).

2.1 (NER con BiLSTMS)

En ésta primera parte, la idea es entrenar dos arquitecturas

- 1) BiLSTM + CRF +fastext
- 2) BILSTM + CRF + CNN +fastatex

Descarga de Fasttext:

python

```
1 EMBED_FILE = 'cc.es.300.vec'
2 if not os.path.exists(EMBED_FILE):
3     print("📄 Descargando FastText español...")
4     !curl -o cc.es.300.vec.gz --compressed https://dl.fbaipublicfiles.com/fasttext/vectors-crawl/cc.es.3
5     !gunzip -f cc.es.300.vec.gz
```

El entrenamiento se realizará sobre el dataset [Conll2002](#), partiendo del modelo base [Bilstm-base](#) + [Conll2002](#). El objetivo es mejorar progresivamente el rendimiento de las tres arquitecturas de LSTM (contando el modelo base [Bilstm-base](#)). Por ejemplo, para el modelo [Bilstm-base](#) el f1-score es de 64.3%; a continuación se muestran las métricas de desempeño sobre los datos de prueba o testeo de [Conll2002](#).

```
*** =====
📊 BASELINE NER (Sin FastText)
F1-score: 64.3%
Precision: 62.7%
Recall: 66.1%

📄 Reporte por entidad:
      precision    recall  f1-score   support

      LOC         0.66      0.78      0.72      1084
      MISC         0.23      0.26      0.25       340
      ORG         0.70      0.61      0.65      1400
      PER         0.65      0.76      0.70       735

  micro avg       0.63      0.66      0.64      3559
  macro avg       0.56      0.60      0.58      3559
 weighted avg     0.63      0.66      0.64      3559
```

En tu notebook [BiLSTM-CRF+CNN](#), la idea es que el CNN (Convolutional Neural Network) actúe como un extractor de rasgos locales para enriquecer las representaciones de cada palabra en un vector de tamaño fijo, capturando así su estructura morfológica. Este vector debe concatenarse con el *embedding* de la palabra completa y alimentar el BiLSTM. Como se observa en la figura del código, el vector de caracteres se suma al vector de embeddings de palabras. Para cada palabra se combina el *embeddings* de Fasttext con el *embedding* de caracteres.

```
# Embeddings de palabra
self.word_emb = nn.Embedding.from_pretrained(
    torch.tensor(emb_matrix), freeze=False, padding_idx=0
)

# CharCNN
self.char_cnn = CharCNN(char_vocab_size, char_emb_dim,
num_filters=30,
```

```

        filter_sizes=[3,4,5])

char_out_dim = 30 * 3 # 3 filtros * 30 canales cada uno

# Dimensión total de entrada
total_input_dim = emb_dim + char_out_dim

```

2.2 (NER con LLMs)

Para el *finetuning* de los modelos [BETO](#) y [XML-Roberta](#) sobre el dataset [prostate](#), se debe tomar como referencia el entrenamiento del modelo base de [bert-base-cased](#) sobre el dataset [Conll2002](#) (rendimiento del modelo es del 75% a 6 épocas, usando 8GB de GPU en Colab).

 [3126/3126 04:20, Epoch 6/6]

Epoch	Training Loss	Validation Loss	Precision	Recall	F1	Accuracy
1	0.176400	0.186334	0.667778	0.702103	0.684510	0.945989
2	0.118700	0.164814	0.701423	0.737150	0.718843	0.953618
3	0.085500	0.155191	0.722393	0.767290	0.744165	0.956186
4	0.068000	0.161928	0.730803	0.769393	0.749602	0.957575
5	0.053100	0.164065	0.743068	0.776402	0.759369	0.959382
6	0.042800	0.172161	0.744056	0.775000	0.759213	0.959211

Tenga en cuenta que para el afinamiento de [BETO](#) y [XML-Roberta](#) se deben implementar los siguientes pasos:

1. Crear su token de autenticación del sitio web de [Hugging Face](#) para subir el mejor modelo entrenado.
2. Implementar la fase de preprocesamiento y alineamiento del tokenizador, realizar el padding o relleno de manera dinámica usando la librería **DataCollatorWithPadding**.
3. Implementar el modelo de clasificación usando la librería **AutoModelForTokenClassification**.
4. Definir la estrategia de entrenamiento usando **TrainingArguments**, parametrizando completamente el modelo. Probar *batch sizes* de 8, 16 y 32 con 6-10 épocas.
5. Implementar el **Trainer** del modelo, definiendo los conjuntos de entrenamiento y validación y ejecutando el entrenamiento de ambos modelos. Finalmente, subir el modelo a [Hugging Face](#).

Punto 3. NER con Prompt Engineering y LLMs Generativos

En esta tarea, se debe diseñar un *prompt* para cada uno de los siguientes modelos generativos: [Mistral-7B-v0.2](#), [Gemma](#), [Llama3-2-3B-instruct](#) y [Deepseek](#). El objetivo es extraer una lista de entidades de un texto clínico referente al cáncer de próstata:

- Edad y sexo del paciente
- Biomarcadores
- Tipo de cáncer
- Cirugías
- Edad del paciente
- Diagnóstico
- Hallazgos de imágenes
- Exámenes realizados
- Antecedentes médicos
- Resultados de laboratorios
- Tratamiento y medicación.

Como un ejemplo, revise el modelo [Mistral-7B-v0.2](#), que reconoce entidades nombradas al estilo de [Conll2002](#). Importante tener en cuenta que, al usar *prompt engineering*, **no se realiza afinamiento**; el modelo se guía únicamente mediante instrucciones textuales. Aunque nativamente algunos LLMs (como [Mistral-7B-v0.2](#)) incluyen etiquetas de instrucción (`[INST] ... [/INST]`), los componentes que se deben definir para la extracción son:

1. Llamado al modelo,
2. Cuantización del modelo para ejecutarlo en GPU de Colab,
3. Texto o argumento del *pipeline*,
4. API de conexión a [Hugging Face](#),
5. *Prompt*,
6. Texto de entrada,
7. *Chat template* específico del LLM para realizar la extracción.

Los siguientes son los chat template de los modelos

Table  			
Característica	Llama-3.2	Gemma-2	DeepSeek-LLM
Formato chat	<code>< start_header_id >...</code>	<code><start_of_turn>...</code>	<code>< User >...< Assistant >:</code>
System prompt	Header <code>system</code>	En turno user	En el turno del user
End turn	<code>< eot_id ></code>	<code><end_of_turn></code>	<code>< EOT ></code>
trust_remote_code	No	No	Sí (requerido)
dtype recomendado	<code>float16</code>	<code>bfloat16</code>	<code>bfloat16</code>
Contexto	128K	8K	16K

A continuación, el texto del cual se extraerán las entidades (deberían ser similares a las del dataset de [prostata](#) extraídas con clasificadores BERT):

Paciente masculino de 72 años con antecedentes médicos de hipertensión arterial (HTA). Actualmente presenta un tumor prostático bilateral confirmado mediante biopsia. El diagnóstico histológico reveló un adenocarcinoma de próstata. El puntaje de Gleason reportado fue 3+3, lo que indica un cáncer de bajo grado. El PSA más alto registrado en la historia clínica fue de 9,9 ng/dL. La resonancia magnética no evidenció lesiones metastásicas ni afectación de la cápsula prostática. Las vesículas seminales se observaron normales en las imágenes por resonancia. No se detectaron adenopatías ni lesiones óseas sospechosas. La gammagrafía ósea fue negativa para metástasis. El cuadro clínico sugiere una neoplasia localizada de bajo riesgo.